NECRO

Project Documentation

RSS, resurrected with AI

Kiroween 2026 · Category: Resurrection

Mowlid Haibe · a Faafi project

Version 1.0 · November 2026

RSS, resurrected with AI. A Kiroween 2026 project by Mowlid Haibe · a Faafi project

Hackathon	Kiroween 2026 (Devpost), Oct 30 to Dec 4, 2026
Deadline	**Dec 4, 2026, 2:00 PM PT = Dec 5, 2026, 6:00 AM Malaysia time**. Target: submit Dec 3
Category	**Resurrection**
Bonus prizes	Best Student Project (Resurrection), Best Startup Project, Best Demo, Bonus Blog, Social Blitz
Repo	`github.com/mawlid1431/necro` (public, MIT licence)
Built with	Kiro (IDE, CLI), spec-driven development

Contents

1. The story
2. Problem and audience
3. Features
4. How it works (user journey)
5. Tech stack
6. Architecture
7. Data model
8. Feed engine
9. AI integration
10. MCP server (talk to the dead)
11. Screens and UI
12. Security and privacy
13. Testing
14. How Kiro is used
15. Environment variables
16. Project structure
17. Delivery plan
18. Submission checklist
19. Demo video script
20. Risks and decisions

1. The story

In 2013 the most popular RSS reader in the world was shut down, and RSS itself was left for dead. Yet RSS never really died: almost every blog, newsroom, podcast and YouTube channel still publishes a feed. What died was a good way to *use* it.

Necro brings RSS back to life with today's technology: an AI that reads with you, ranks what matters, and turns what you read into posts you can share. You can even talk to your feeds from Claude through MCP.

> **Trademark note.** Tell the story in writing (description, blog). In the **video, logo and images**, say "RSS" and "classic feed readers" instead of naming the old Google product, because the rules forbid third-party trademarks in the video without permission.

Why it is the first piece of Faafi: Faafi (Somali for "spread") is a workspace for publishing to every channel. Necro is its first module: the *input* side, turning what you read into content you spread.

2. Problem and audience

Who	Pain today	What Necro gives them
Creators and marketers	Must read a lot to post well; social feeds are noisy and algorithmic	A calm, chronological reader plus AI drafts ready to share
Students and researchers	Follow many blogs, papers and news sites in tabs	One place, AI summaries, saved highlights
Former RSS users	Lost their favourite reader; old exports sit unused	Import the old OPML export and everything comes back
AI power users	Their assistant cannot read their sources	An MCP server so Claude can browse and draft from their feeds

Judging angle (Potential Impact): a real, large audience (anyone who reads to create), an easy start (paste a link or upload one file), accessible UI (contrast-checked colours, keyboard navigation, reduced-motion mode).

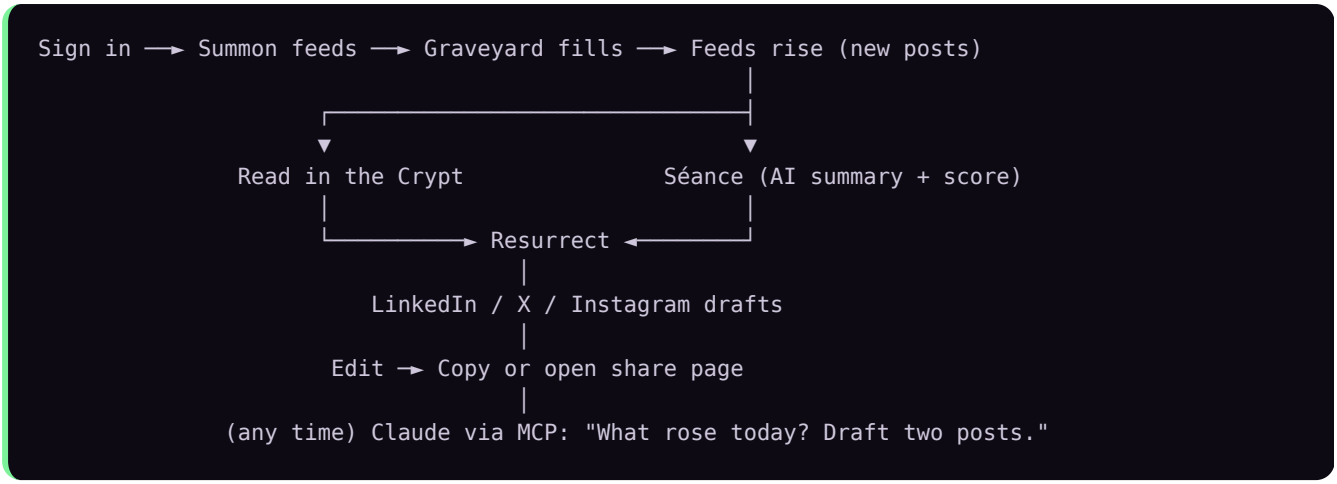
3. Features

The feature names follow the theme, and each has a plain-language subtitle in the UI.

#	Feature	Plain meaning	MVP
1	**Summon**	Add feeds: paste a URL, a website (auto-discover its feed) or upload OPML	✅
2	**The Graveyard**	Your feeds as tombstones; new posts make a tombstone "rise"	✅
3	**The Crypt (reader)**	Clean, chronological reading view with read/unread, star, search	✅
4	**Séance**	AI summary (3 bullets) and a relevance score for each article	✅
5	**Resurrect**	Turn an article into LinkedIn, X and Instagram drafts in your voice	✅
6	**Talk to the dead**	MCP server: Claude can list, read, summarise and draft from your feeds	✅
7	**Feed Doctor**	Detects broken or moved feeds and suggests fixes	✅ (basic)
8	**Haunted digest**	Daily email or page: "What rose last night"	Stretch
9	**Share**	Copy draft, or open the platform's own share/compose page with the text filled in	✅

Out of scope for the hackathon: posting directly through social APIs (needs platform app reviews). Share links and copy are reliable and need no review. Direct publishing comes later through Faafi.

4. How it works (user journey)



Step by step

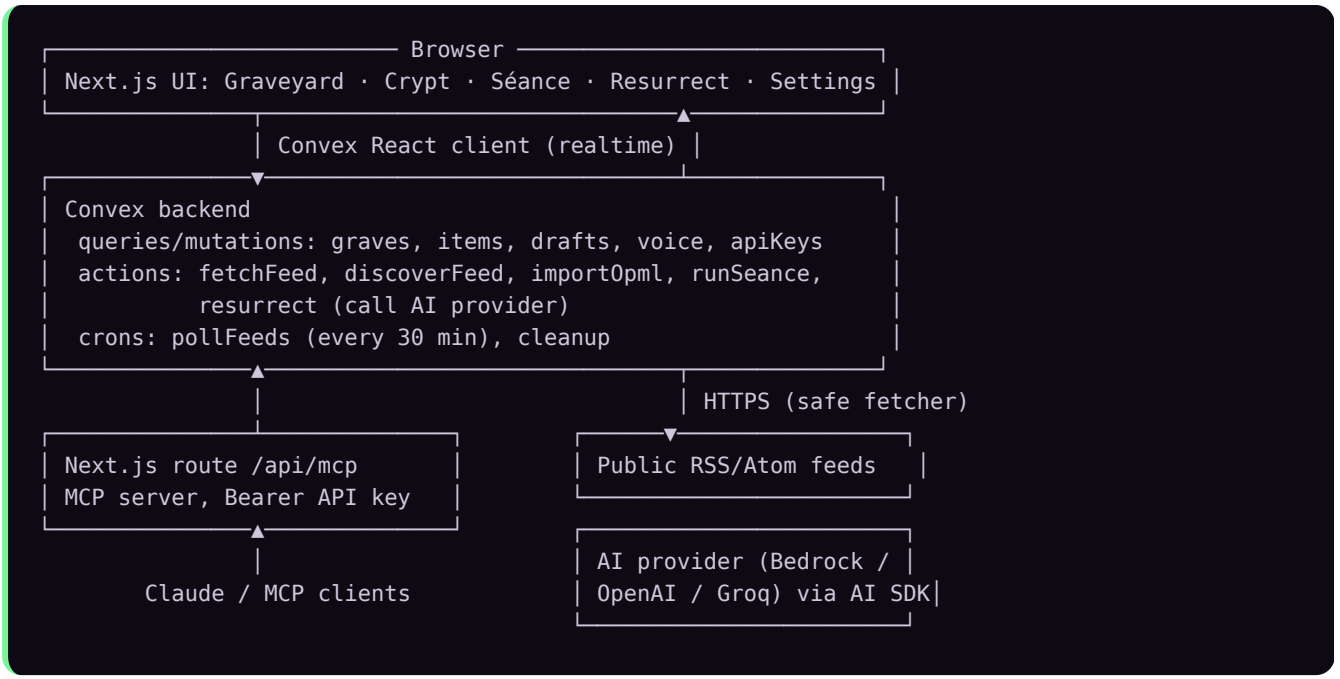
1. **Sign in** with Clerk (Google or email).
2. **Summon**: paste `https://example.com` or a feed URL, or upload an `.opml` file. Necro finds the feed, validates it and creates a tombstone.
3. **Rise**: a background job checks each feed on a schedule (default every 30 minutes, respecting ETag and Last-Modified). New items "rise" with a soft glow animation.
4. **Read** in the Crypt. Mark read, star or search.
5. **Séance** runs on new items: a 3–bullet summary and a 0 to 100 relevance score based on the user's interests.
6. **Resurrect**: pick platforms; AI writes drafts in the user's saved voice. The user edits them.
7. **Share**: copy text or open the platform's compose page.
8. **Talk to the dead**: in Claude, the user connects Necro's MCP server with a personal key and asks in plain language.

5. Tech stack

Layer	Choice	Why
Runtime and package manager	**Bun**	Fast installs and scripts
Framework	**Next.js 16** (App Router) + **TypeScript** (strict)	Full-stack React, server actions, route handlers for MCP
Styling	**Tailwind CSS v4** + **shadcn/ui**	Fast, consistent, themeable
Animation	**Motion** (Framer Motion)	Rise, fog and glow effects, with reduced-motion support
Database and backend	**Convex**	Realtime (tombstones rise live), scheduled jobs (cron), actions for fetching
Auth	**Clerk**	Sign-in in minutes
AI SDK	**Vercel AI SDK** (`ai`)	One API for many model providers, structured output with Zod
AI provider	**Amazon Bedrock** (Anthropic Claude model) by default; **OpenAI** or **Groq** as alternatives	Bedrock fits an AWS-judged hackathon; provider is swappable by env var
Feed parsing	**fast-xml-parser** + own normaliser	Handles RSS 0.9×/1.0/2.0 and Atom
HTML safety	**sanitize-html**	Clean article HTML before display
Validation	**Zod**	Inputs, AI output, MCP tool arguments
MCP	**@modelcontextprotocol/sdk** (Streamable HTTP) in a Next.js route	Claude and other clients connect remotely
Testing	**Vitest**, **fast-check** (property-based), **Playwright** (end-to-end), **convex-test**	Matches Kiro's PBT feature
Hosting	**Vercel** (app) + **Convex Cloud** (backend)	Free tiers, fast deploys
Icons	**Lucide**	Clean outline icons

Pin exact versions in `package.json`. Check each library's current docs at build time; APIs change.

6. Architecture



Rules

- All business logic lives in Convex functions. The UI and the MCP route call the **same** functions, so behaviour is identical.
- Every record carries `userId`. Every query checks it. One user can never see another user's data.
- Network fetching happens only in Convex **actions** through one safe fetcher (section 12).

7. Data model

Convex tables (all with `userId`, indexed):

Table	Key fields	Notes
`users`	clerkId, name, email, interests[], createdAt	Synced on first sign-in
`graves` (feeds)	userId, feedUrl, siteUrl, title, iconUrl, status (`alive`, `dormant`, `broken`), etag, lastModified, lastFetchedAt, nextFetchAt, errorCount, lastError	Unique per user + feedUrl
`items`	userId, graveId, guid, title, link, author, publishedAt, contentHtml (sanitised), excerpt, isRead, isStarred, risenAt	Unique per grave + guid (fallback: hash of link + title)
`seances`	userId, itemId, summary[3], score (0 to 100), model, createdAt	One per item
`drafts`	userId, itemId, platform (`linkedin`, `x`, `instagram`), text, hashtags[], status (`draft`, `copied`, `shared`), updatedAt	Editable
`voice`	userId, tone, audience, wordsToUse[], wordsToAvoid[], samplePosts[]	Used by Resurrect
`apiKeys`	userId, name, hashedKey, prefix, lastUsedAt, revokedAt	For MCP; store only a hash

Feed statuses

Status	Meaning	UI
alive	Fetched OK recently	Tombstone glows when items rise
dormant	No new items for 90 days	Mossy tombstone, "Sleeping"
broken	3+ failed fetches	Cracked tombstone, Feed Doctor offers a fix

8. Feed engine

8.1 Summon (adding feeds)

1. Normalise the input URL (add `https://` if missing).
2. If it is a web page, **discover** feeds via `<link rel="alternate" type="application/rss+xml|atom+xml">`, then common paths (`/feed`, `/rss`, `/atom.xml`, `/feed.xml`).
3. Fetch with the safe fetcher, parse, validate there is at least a title or items.
4. Create the grave and insert the first items.

OPML import: parse `<outline xmlUrl="...">` (nested folders allowed), deduplicate, add up to 200 feeds, report results ("42 summoned, 3 could not be reached").

8.2 Parsing

- Support RSS 0.91/0.92/2.0, RSS 1.0 (RDF) and Atom 1.0.
- Normalise to one `Item` shape: `{ guid, title, link, author, publishedAt, contentHtml, excerpt }`.
- Dates: parse RFC 822 and ISO 8601; fall back to fetch time if missing or invalid.
- Missing guid: use `link`, else a hash of title + date.
- Never throw on one bad item: skip it and log a warning.

8.3 Polling (the rise)

- A Convex cron runs every 5 minutes and fetches graves whose `nextFetchAt` has passed (max 25 per run).
- Send `If-None-Match` / `If-Modified-Since`; a `304` means nothing new.
- Back off on errors: 30 min, 2 h, 6 h, 24 h. After 3 failures mark `broken`.
- Identify politely: `User-Agent: NecroBot/1.0 (+https://[your-domain]/bot)`.

8.4 Feed Doctor

For `broken` graves: re-run discovery on `siteUrl`, try `http↔https`, follow permanent redirects, and suggest the new URL. The user confirms the fix.

9. AI INTEGRATION

9.1 Provider setup (Vercel AI SDK)

One small module `lib/ai.ts` returns the model from environment variables, so you can switch providers without code changes:

```
// lib/ai.ts (sketch)
import { bedrock } from "@ai-sdk/amazon-bedrock";
import { openai } from "@ai-sdk/openai";
import { groq } from "@ai-sdk/groq";

export function getModel() {
  const id = process.env.AI_MODEL!;           // e.g. a Claude model ID on Bedrock
  switch (process.env.AI_PROVIDER) {
    case "bedrock": return bedrock(id);
    case "openai":  return openai(id);
    case "groq":    return groq(id);
    default: throw new Error("Set AI_PROVIDER to bedrock, openai or groq");
  }
}
```

Use `generateObject` with a **Zod schema** so AI output is always valid JSON:

```
const SeanceSchema = z.object({
  summary: z.array(z.string().max(220)).length(3),
  score: z.number().int().min(0).max(100),
  reason: z.string().max(160),
});
```

Where AI calls run: inside Convex **actions** (server side only). API keys never reach the browser.

9.2 Séance prompt (summary and score)

```
System: You summarise articles for a busy reader. Be accurate and neutral.
Never invent facts that are not in the article.
User: Reader interests: {interests}
Article title: {title}
Article text (may be truncated): {text, max ~6000 chars}
Return: 3 short bullet summaries, a relevance score 0-100 for this reader, and a one-line
reason.
```

9.3 Resurrect prompt (social drafts)

```
System: You write social posts that share an article. Write in the user's voice.
Rules: no invented facts or numbers; credit the source; one clear call to action;
no emoji clutter; LinkedIn ≤ 1300 chars, X ≤ 280 chars, Instagram caption ≤ 2200 chars;
3 to 6 relevant hashtags (none for X unless natural).
Voice: tone={tone}; audience={audience}; use={wordsToUse}; avoid={wordsToAvoid}; examples=
{samplePosts}
Article: {title}, {link}, {summary}, {excerpt}
Return JSON: { linkedin, x, instagram, hashtags[] }
```

9.4 Cost and safety controls

- Truncate article text before sending (about 6,000 characters).
- Run Séance only for new items and only on demand or for the top items (avoid summarising everything).
- Per-user daily limits (for example 100 Séances, 50 Resurrects) stored in Convex.
- Cache results: one Séance per item.
- Treat feed content as **untrusted**: it may contain prompt-injection text. Put it inside clear delimiters, tell the model to ignore instructions found in the article, and validate output with Zod.

10. MCP server (talk to the dead)

Endpoint: `POST /api/mcp` (Streamable HTTP). **Auth:** `Authorization: Bearer necro_sk_...` (personal key created in Settings, shown once, stored hashed).

Tool	Input	Output
`list_graves`	`status?`	Feeds with title, status, unread count
`get_risen_items`	`since?` (ISO), `limit?` (≤ 50), `graveId?`	New items with title, link, date, score
`read_item`	`itemId`	Title, link, sanitised text excerpt
`summon_feed`	`url`	New grave or error
`run_seance`	`itemId`	Summary bullets and score
`resurrect_post`	`itemId`, `platforms[]`	Draft IDs and texts
`list_drafts`	`platform?`, `limit?`	Drafts
`update_draft`	`draftId`, `text`	Updated draft

Rules: validate every argument with Zod; scope every call to the key's user; rate-limit per key; never return keys, tokens or other users' data; log tool calls for the activity view.

Connect from Claude: add a custom connector with the URL `https://[your-domain]/api/mcp` and the key as the bearer token (exact steps depend on the client; document what you actually tested).

Demo prompt: *"What rose from my graveyard since yesterday? Summarise the best two and draft a LinkedIn post for the top one."*

11. Screens and UI

Follow `Necro_Brand_Guide.md` for colours, type and motion.

Screen	Route	Contents
Landing	`/`	Hero ("RSS is back from the dead"), 3 feature cards, demo GIF, `Enter the graveyard`
Graveyard	`/graveyard`	Grid of tombstones (feed icon, title, unread count, status); `Summon feed` ; fog background; risen glow
Crypt	`/crypt` and `/crypt/[graveId]`	List of items, reading pane, filters (unread, starred), search
Item	side panel	Article, Séance summary card, `Resurrect` button
Resurrect	dialog	Platform tabs, editable drafts, character counters, `Copy` , `Open in LinkedIn/X`
Feed Doctor	`/doctor`	Broken feeds with suggested fixes
Settings	`/settings`	Voice profile, interests, MCP keys (create, copy once, revoke), sound on/off, motion on/off

States everywhere: loading (skeleton tombstones), empty ("The graveyard is quiet. Summon your first feed."), error ("This feed did not answer. Feed Doctor can help."), offline.

12. Security and privacy

Risk	Control
SSRF (fetching internal addresses via a feed URL)	Only `http` / `https`; resolve DNS and **block private, loopback, link-local and metadata IPs** (10/8, 172.16/12, 192.168/16, 127/8, 169.254/16, ::1, fc00::/7); re-check after redirects; max 5 redirects
Huge or slow responses	10 s timeout, 5 MB max body, only XML/HTML content types
Malicious HTML in articles	Sanitise with an allowlist; strip scripts, iframes, event handlers; images via `referrerpolicy=no-referrer`
Prompt injection from feeds	Delimit untrusted content, instruct the model to ignore it, validate output
Cross-user access	`userId` checks in every Convex function; tests prove it
MCP key theft	Store only hashes; show once; revoke; rate limit
AI keys	Server-side only (Convex env), never in the client bundle
Malicious OPML (XML bombs)	Disable entity expansion in the parser; size limit

13. Testing

Type	Tool	What
Unit	Vitest	URL normaliser, date parser, guid fallback, OPML parser
Property-based	**fast-check**	Parser never throws on random/mutated XML; normalised items always have a title or link; OPML round-trip (export → import gives the same feeds); SSRF guard rejects every generated private IP
Backend	convex-test	User isolation, polling backoff, duplicate item prevention
End-to-end	Playwright	Sign in → summon → see items → resurrect → copy
Fixtures	`tests/fixtures/feeds/*.xml`	Real-world samples: RSS 2.0, Atom, RSS 1.0, broken encodings, missing dates

Mention PBT findings in the Kiro write-up: judges ask whether PBT caught issues unit tests missed.

14. How Kiro is used

This is a judging criterion (**Technological Implementation**). Use each feature for real and record what happened, so the write-up is honest and specific.

Kiro feature	Plan
Specs	One spec per feature in `.kiro/specs/<feature>/` with `requirements.md` (EARS format), `design.md`, `tasks.md`: summon, graveyard, crypt, seance, resurrect, mcp-server, feed-doctor
Steering	`.kiro/steering/product.md`, `tech.md`, `structure.md`, `brand.md`, `security.md`, `ai-prompts.md`
Agent hooks	On save of `*.test.ts` → run tests; on change in `convex/schema.ts` → update the data model doc; on new component → accessibility check; before commit → lint and typecheck
MCP (during development)	Connect useful MCP servers in Kiro (for example docs lookup or browser testing) and describe how they helped
MCP (in the product)	Necro's own MCP server
Property-based testing	Feed parser and SSRF guard (section 13)
Custom agents	"Feed Doctor" agent for debugging feeds; "Copy Editor" agent for UI text in brand voice
Kiro powers	Try relevant powers; note which ones and what they added
IDE vs CLI	IDE for specs and building; CLI for scripted tasks; compare in the write-up
Spec vs vibe coding	Vibe-code one small thing (e.g. the fog animation) and compare with spec-driven features

Commit the whole `/.kiro` folder. Never add it to `.gitignore`.

15. Environment variables

`.env.example` (commit this, never the real `.env.local`):

```
# Convex
CONVEX_DEPLOYMENT=
NEXT_PUBLIC_CONVEX_URL=

# Clerk
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=
CLERK_SECRET_KEY=
CLERK_JWT_ISSUER_DOMAIN=

# AI (choose one provider)
AI_PROVIDER=bedrock           # bedrock | openai | groq
AI_MODEL=                     # model ID for that provider
AWS_REGION=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
OPENAI_API_KEY=
GROQ_API_KEY=

# App
NEXT_PUBLIC_APP_URL=http://localhost:3000
NECRO_BOT_CONTACT_URL=
```

AI and AWS keys go in **Convex environment variables** (server side), not `NEXT_PUBLIC_*`.

16. Project structure

```
necro/
├─ .kiro/
│  ├─ specs/{summon,graveyard,crypt,seance,resurrect,mcp-server,feed-doctor}/
│  │    requirements.md · design.md · tasks.md
│  ├─ steering/  product.md · tech.md · structure.md · brand.md · security.md · ai-prompts.md
│  └─ hooks/     (created in Kiro)
├─ app/
│  ├─ (marketing)/page.tsx
│  ├─ graveyard/ · crypt/ · doctor/ · settings/
│  └─ api/mcp/route.ts
├─ components/   graveyard/ · crypt/ · resurrect/ · ui/ (shadcn)
├─ convex/       schema.ts · graves.ts · items.ts · seance.ts · resurrect.ts · crons.ts ·
fetcher.ts
├─ lib/          ai.ts · feeds/{parse,discover,opml,normalize}.ts · security/ssrf.ts ·
sanitize.ts
├─ public/       brand assets (from Necro_Brand_Assets)
├─ tests/        unit/ · pbt/ · e2e/ · fixtures/feeds/
├─ LICENSE (MIT) · README.md · .env.example
```

17. Delivery plan

Today is Nov 1. Deadline Dec 5, 6:00 AM Malaysia time.

Week	Dates	Deliverables
1	Nov 1 to 8	Join hackathon, request **4,000 Kiro credits** (by Dec 1), Discord. Repo + MIT licence + `.kiro` steering. Specs for Summon and Crypt. Feed parser + PBT. Auth + schema
2	Nov 9 to 15	Graveyard UI, polling cron, OPML import, Crypt reader. Attend **Devpost build session Nov 11**
3	Nov 16 to 22	Séance + Resurrect (AI), voice profile, share links. Hooks running. Deploy to Vercel
4	Nov 23 to 29	MCP server + Claude demo, Feed Doctor, security hardening, Playwright tests, spooky polish (fog, glow, sound)
5	Nov 30 to Dec 3	Freeze features. Video, write-up, README, blog post, social post, Faafi website (for startup prize). **Submit Dec 3**

18. Submission checklist

- [ ] Category selected: **Resurrection**; bonus prizes ticked: Student, Startup, Demo, Blog, Social Blitz
- [ ] Public **GitHub** repo with all code, assets and setup instructions
- [ ] **MIT** licence visible in the repo's About section
- [ ] `/.kiro` at the root with specs, hooks, steering, **not gitignored**
- [ ] Live app URL; test login in the private submission field
- [ ] Text description of features
- [ ] Demo video **under 3 minutes**, public on YouTube, no copyrighted music, no third-party trademarks
- [ ] Kiro write-up covering every feature in section 14
- [ ] Student proof: LinkedIn, student email, year, degree, school (AIU)
- [ ] Startup: confirm founder, LinkedIn, Faafi website URL
- [ ] Blog post (dev.to or Hashnode) stating it is written for the Kiroween 2026 hackathon
- [ ] Social post starting with **#KiroweenHackathon26**
- [ ] All code written between Oct 30 and Dec 4, 2026

19. Demo video script

Under 3:00. Record the real app. Royalty-free sound only.

Time	Scene	Voice-over
0:00 to 0:15	Fog, tombstone logo rises	"RSS was left for dead. Millions of feeds are still out there, with nowhere good to read them. This is Necro."
0:15 to 0:40	Upload OPML, graveyard fills	"Bring back your old feeds in one file. Each feed becomes a tombstone."
0:40 to 1:05	New items rise, Crypt reader	"When something new is published, it rises. Read it in one calm place."
1:05 to 1:30	Séance summary + score	"Séance summarises each article and ranks what matters to you."
1:30 to 1:55	Resurrect drafts, copy, open LinkedIn	"Resurrect turns it into posts for LinkedIn, X and Instagram, in your voice."
1:55 to 2:25	Claude with MCP	"And you can talk to the dead. Ask Claude what rose today and it drafts from your own feeds."
2:25 to 2:50	Kiro: specs, hooks, PBT	"Built with Kiro: spec-driven features, hooks that test as I code, property-based tests that caught real parser bugs."
2:50 to 3:00	Logo, URL	"Necro. RSS, resurrected."

20. Risks and decisions

Risk	Plan
Running out of time	MVP = features 1 to 6. Feed Doctor basic. Digest is a stretch goal
AI costs	Limits, caching, truncation; use free credits; Groq as a cheap fallback
Claude connector setup differs by plan	Test early (week 4); if blocked, demo with another MCP client and say so honestly
Platform posting APIs	Out of scope; use copy and official share links
Feeds blocking bots	Polite User-Agent, conditional requests, backoff
Trademarks	No Google product name or logo in video or images

Necro Project Documentation · v1.0 · November 2026